



Machine-Level Programming VI: Advanced Datatypes – Arrays, Structs, and Unions

cs144

Prof. Yanjing Li



Back to Data

▣ Integers

- ★ Unsigned and signed (two's complement), of various sizes

▣ What if you need some grouping of multiple values?

▣ Arrays

- ★ Collection of values, all of same type, indexed by one or more indices
- ★ E.g., $A[i]$ (1D array), $A[i][j]$ (2D array, or matrix)

▣ Structs and unions

- ★ Collections of differently typed values

Today

- ▢ **Arrays**
 - ▣ Single-dimensional arrays
 - ▣ Multi-dimensional arrays
- ▢ **Structs**
- ▢ **Unions**



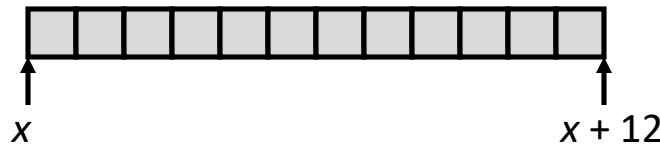
Array Allocation

Basic principle

$T \text{ } \mathbf{A}[L]$; e.g., `int val[5]`

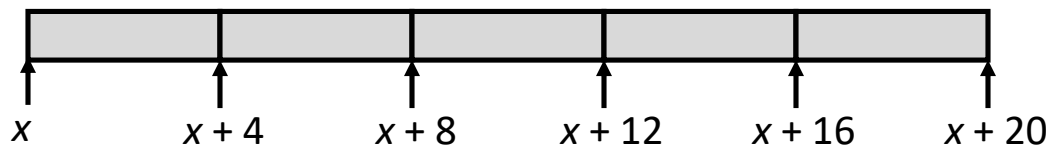
- ★ Array of data type T and length L
- ★ **Contiguously** allocated region of $L * \text{sizeof}(T)$ bytes
- ★ Identifier $\mathbf{A}$ can be used as a pointer to array element 0: Type T^*

`char string[12];`



*Sumps. based on
now by the
data type is*

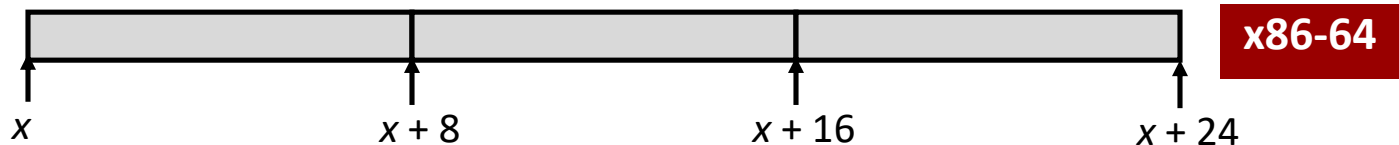
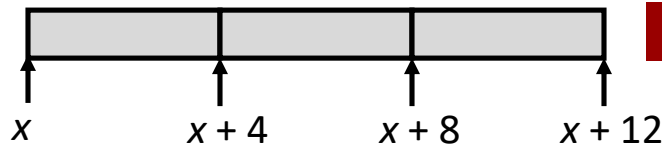
`int val[5];`



`double a[3];`



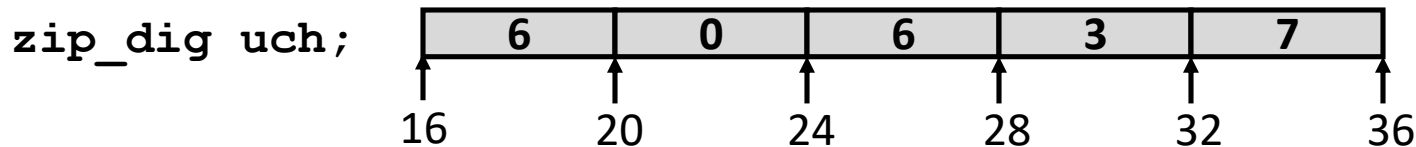
`char *p[3];`





Array Access Example

```
#define ZLEN 5
typedef int zip_dig[ZLEN];
zip_dig uch = { 6, 0, 6, 3, 7 };
```



```
int get_digit(zip_dig z, size_t dig)
{
    return z[dig];
}
```

size_t is unsigned integer in C
(8 bytes on 64-bit machines)

```
# %rdi = z
# %rsi = dig
movl (%rdi,%rsi,4),%eax # z[dig]
```

array going to this

%eax: lower 32 bits of %rax
movl automatically zeros higher 32 bits

- Register %rdi contains starting address of array
- Register %rsi contains array index
- Desired digit at $4 * \%rsi + \%rdi$
- Use scaled indexed memory reference $(\%rdi, \%rsi, 4)$

Today

- ▣ **Arrays**
 - ☒ Single-dimensional arrays
 - ☒ Multi-dimensional arrays
- ▣ **Structs**
- ▣ **Unions**



Multidimensional (Nested) Arrays

□ Declaration

$T \ A[R][C];$

▣ 2D array of data type T

▣ R rows, C columns

□ Array Size

▣ $R * C * K$ bytes

▣ Given that Type T element requires K bytes

$$\begin{bmatrix} A[0][0] & \cdot & \cdot & \cdot & A[0][C-1] \\ \cdot & & & & \cdot \\ \cdot & & & & \cdot \\ \cdot & & & & \cdot \\ A[R-1][0] & \cdot & \cdot & \cdot & A[R-1][C-1] \end{bmatrix}$$

Multidimensional (Nested) Arrays

Declaration

`T A[R][C];`

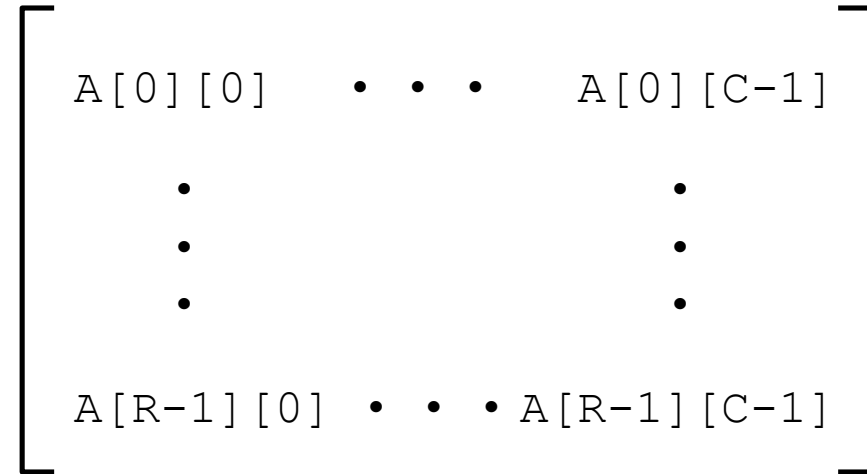
▣ 2D array of data type *T*

▣ *R* rows, *C* columns

Array Size

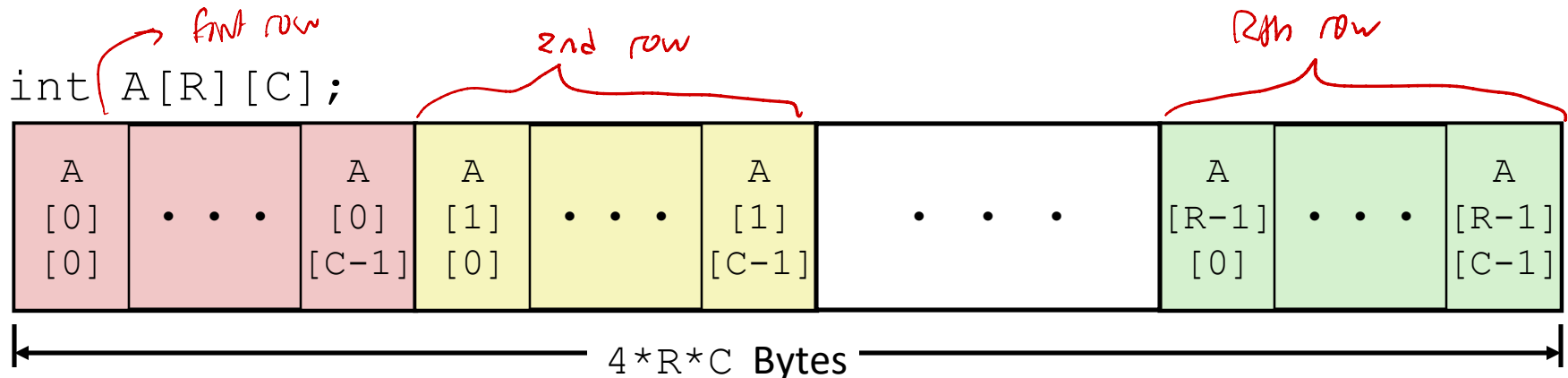
▣ $R * C * K$ bytes

▣ Given that Type *T* element requires *K* bytes



Arrangement

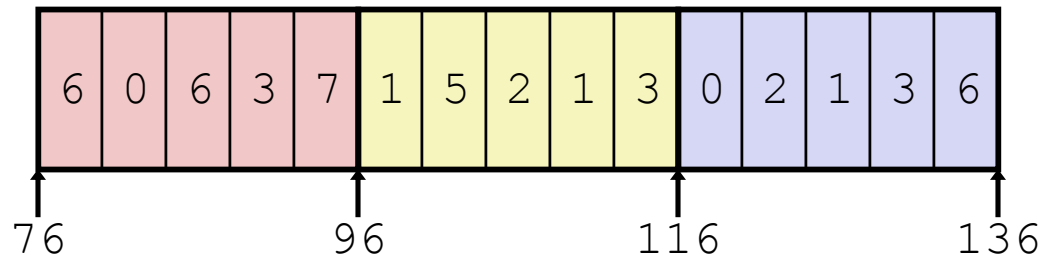
▣ **Row-Major Ordering**



Nested Array Example

```
#define PCOUNT 3
zip_dig univ[PCOUNT] =
    {{6, 0, 6, 3, 7 },
     {1, 5, 2, 1, 3 },
     {0, 2, 1, 3, 6 }};
```

zip_dig
univ[3];



- ▣ “`zip_dig univ[3]`” equivalent to “`int univ[3][5]`”
(remember: `typedef int zip_dig[5]`)
 - ▣ Variable `univ`: array of 3 `zip_dig` elements, allocated *contiguously*
 - ▣ Each `zip_dig` element is an array of 5 `int`’s, allocated *contiguously*
- ▣ “Row-Major” ordering of all elements in memory

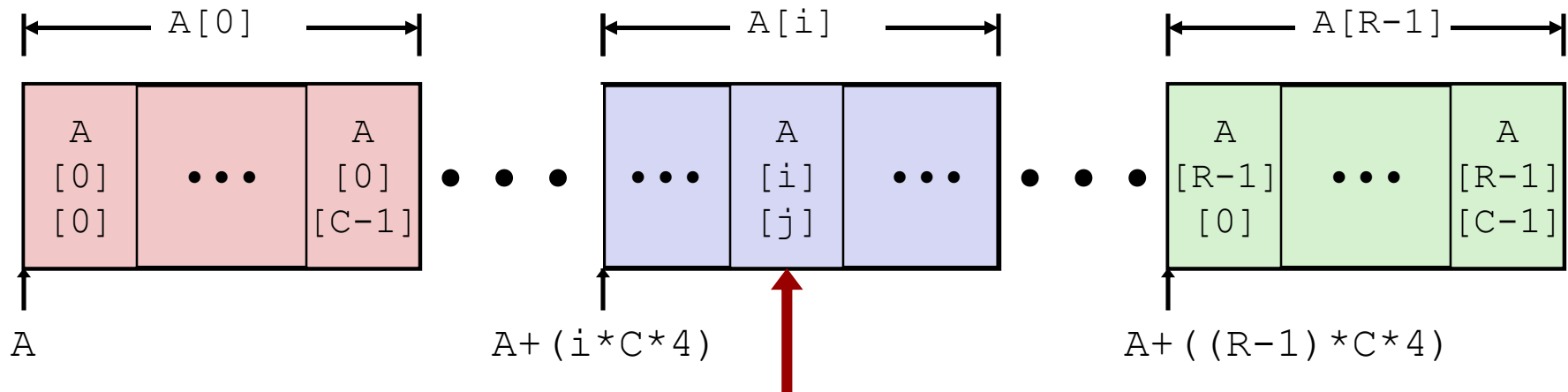
Nested Array *Element* Access

Array Elements

▣ $A[i][j]$ is element of type T , which requires K bytes

▣ Address $A + i * (C * K) + j * K = A + (i * C + j) * K$

```
int A[R][C];
```



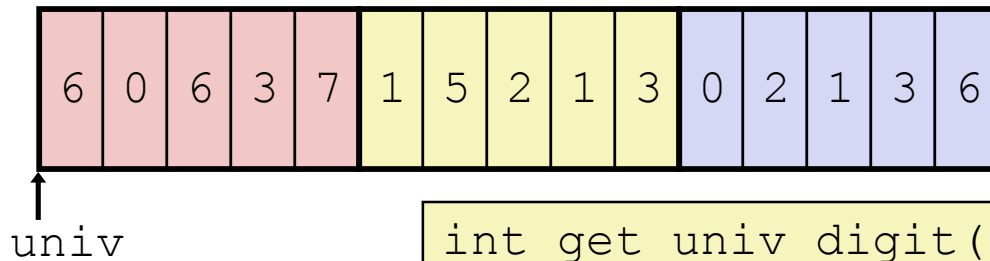
$$A + (i * C * 4) + (j * 4) = A + (i * C + j) * 4$$

skip this many rows
go to the correct column

**Total # of elements
away from A**



Nested Array Element Access Code



```
int get_univ_digit(size_t index, size_t digit)
{
    return univ[index][digit];
}
```

```
# %rdi = index, %rsi = digit, %rdx = univ
leaq    (%rdi,%rdi,4), %rax    # 5*index
addq    %rax, %rsi             # 5*index+digit
movl    (%rdx,%rsi,4), %eax    # Mem[univ + 4*(5*index+digit)]
```

□ Array Elements

▣ `univ[index][digit]` is int

▣ Address: `univ + 4 * (5 * index + digit)`

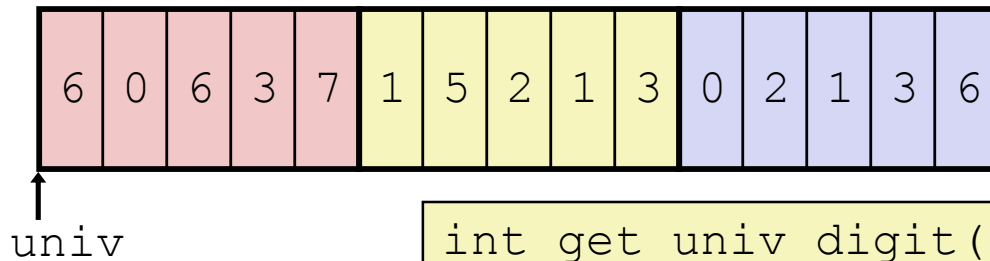
base

*rows
to jump*

col



Nested Array Element Access Code



```
int get_univ_digit(size_t index, size_t digit)
{
    return univ[index][digit];
}
```

```
# %rdi = index, %rsi = digit, %rdx = univ
leaq   (%rdi,%rdi,4), %rax    # 5*index
addq   %rax, %rsi              # 5*index+digit
movl   (%rdx,%rsi,4), %eax     # Mem[univ + 4*(5*index+digit)]
```

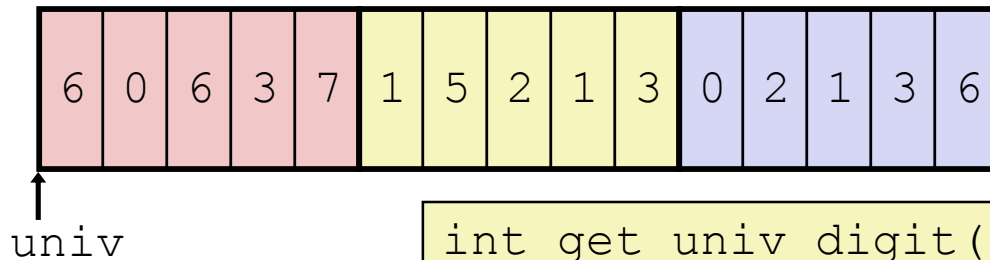
□ Array Elements

▣ `univ[index][digit]` is `int`

▣ Address: `univ + 4*(5*index + digit)`
`= univ + 4*(index+4*index + digit)`



Nested Array Element Access Code



```
int get_univ_digit(size_t index, size_t digit)
{
    return univ[index][digit];
}
```

```
# %rdi = index, %rsi = digit, %rdx = univ
leaq    (%rdi,%rdi,4), %rax    # 5*index
addq    %rax, %rsi           # 5*index+digit
movl    (%rdx,%rsi,4), %eax    # Mem[univ + 4*(5*index+digit)]
```

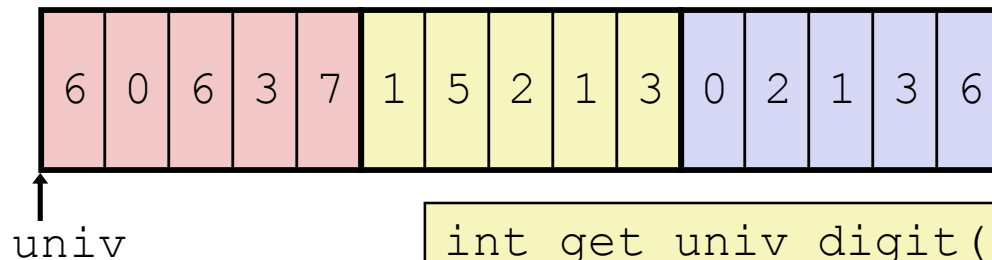
□ Array Elements

▣ `univ[index][digit]` is `int`

▣ Address: `univ + 4*(5*index + digit)`
`= univ + 4*(index+4*index + digit)`



Nested Array Element Access Code



```
int get_univ_digit(size_t index, size_t digit)
{
    return univ[index][digit];
}
```

```
# %rdi = index, %rsi = digit, %rdx = univ
leaq    (%rdi,%rdi,4), %rax    # 5*index
addq    %rax, %rsi            # 5*index+digit
movl    (%rdx,%rsi,4), %eax    # Mem[univ + 4*(5*index+digit)]
```

□ Array Elements

▣ `univ[index][digit]` is `int`

▣ Address: `univ + 4*(5*index + digit)`
= `univ + 4*(index+4*index + digit)`



Today

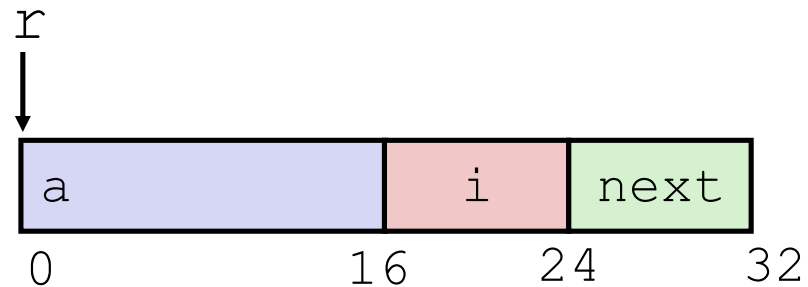
 ~~Arrays~~

 Structs

 Unions

Structure Representation

```
struct rec {  
    int a[4];  
    size_t i;  
    struct rec *next;  
};
```



- Structure represented as block of memory

- ☒ Big enough to hold all of the fields

- Fields ordered according to declaration

- ☒ Even if another ordering could yield a more compact representation

- Compiler determines overall size + positions of fields

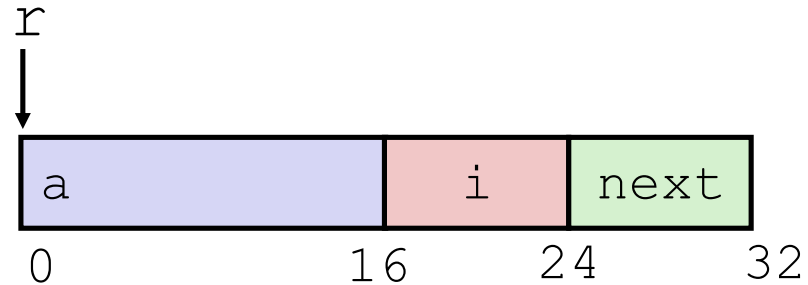
- ☒ Machine-level program has no understanding of the structures in the source code

*initialized in
order, the
compiler should
already know how
much space is needed*



Generating Pointer to Structure Member

```
struct rec {  
    int a[4];  
    size_t i;  
    struct rec *next;  
};
```



Generating Pointer to Array Element

Offset of each structure member determined at compile time

Compute as $r + 4 * \text{index}$

```
int *get_ap  
(struct rec *r, size_t index)  
{  
    return &(r->a[index]);  
}
```

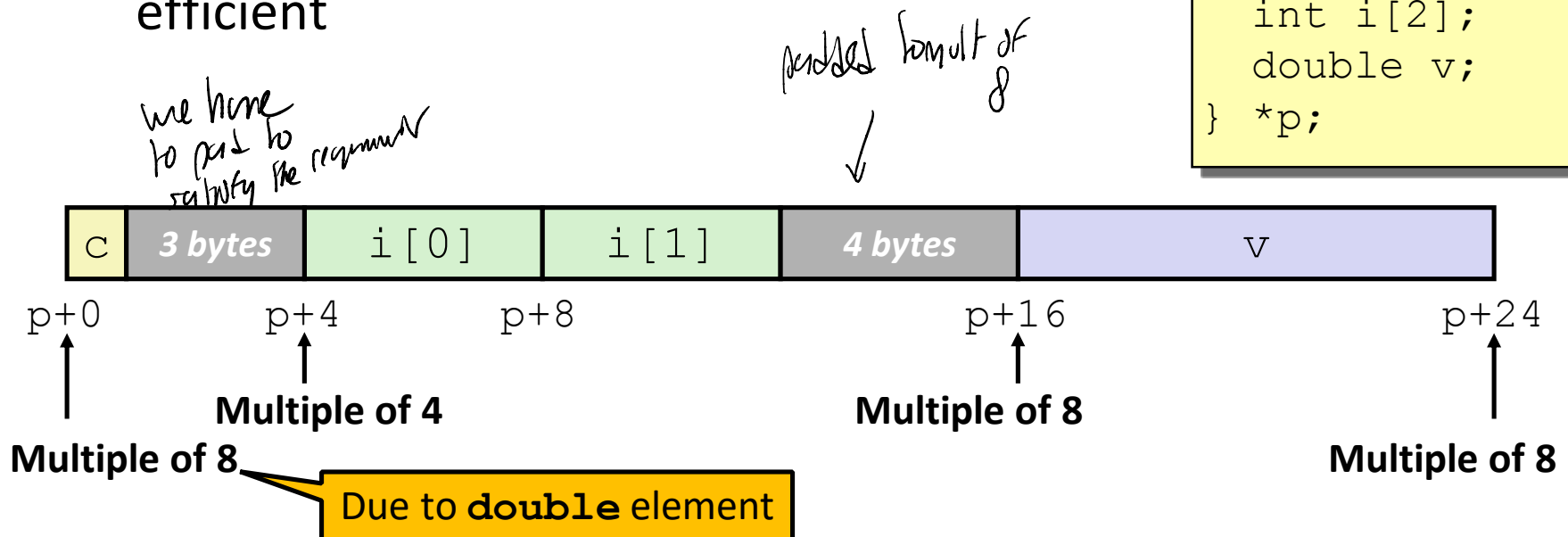
```
# r in %rdi, index in %rsi  
leaq (%rdi,%rsi,4), %rax  
ret
```

The address of struct is the beginning of the array element

Structures & Alignment

- ▣ Primitive data type requires K bytes →
Address must be multiple of K
- ▣ Struct address must be multiple of the largest alignment of any element
- ▣ Compiler follows the alignment rules
- ▣ Align data to make memory accesses more efficient

```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```



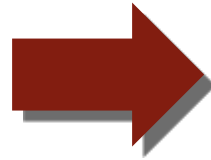
Specific Cases of Alignment (x86-64)

- ▣ 1 byte: **char**, ...
 - ▣ no restrictions on address
- ▣ 2 bytes: **short**, ...
 - ▣ Addr. is a multiple of 2, i.e., lowest 1 bit of address must be 0_2
- ▣ 4 bytes: **int**, **float**, ...
 - ▣ Addr. is a multiple of 4, i.e., lowest 2 bits of address must be 00_2
- ▣ 8 bytes: **double**, **long**, **char ***, ...
 - ▣ Addr. is a multiple of 8, i.e., lowest 3 bits of address must be 000_2
- ▣ 16 bytes: **long double** (GCC on Linux)
 - ▣ Addr. is a multiple of 16, i.e., lowest 4 bits of address must be 0000_2

Saving Space

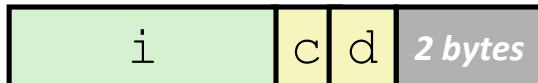
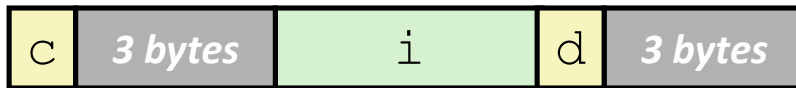
- Put large data types first

```
struct S4 {  
    char c;  
    int i;  
    char d;  
} *p;
```



```
struct S5 {  
    int i;  
    char c;  
    char d;  
} *p;
```

- Effect (12 bytes vs. 8 bytes)



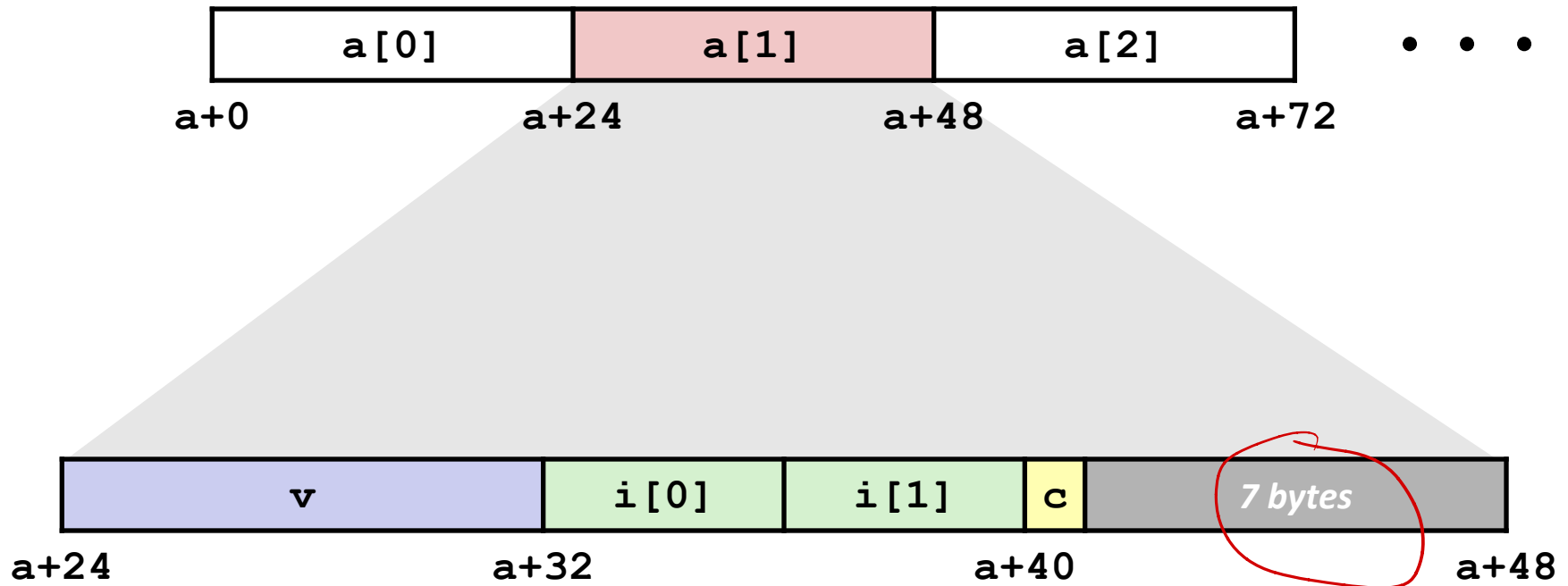


Arrays of Structures

- Satisfy alignment requirement for every element

```
struct S2 {  
    double v;  
    int i[2];  
    char c;  
} a[10];
```

structs must also follow alignment





Today

☒ ~~Arrays~~

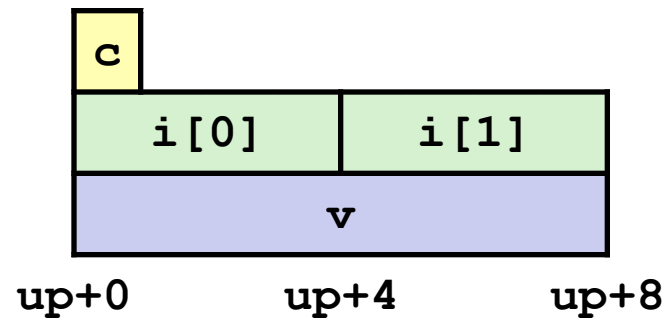
☒ ~~Structs~~

☐ **Unions**

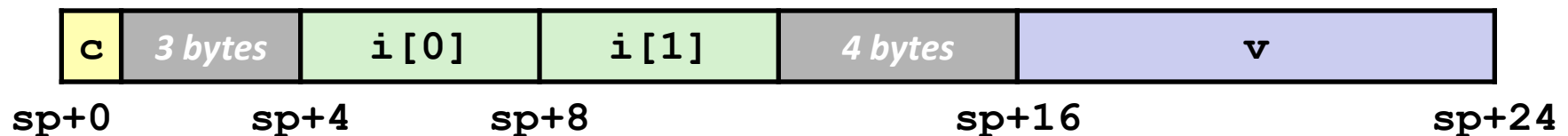
Union Allocation

- Allocate according to largest element
- Can only use one field at a time

```
union U1 {  
    char c;  
    int i[2];  
    double v;  
} *up;
```



```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *sp;
```



Summary

▣ **Arrays in C**

- ✦ Contiguous allocation of memory
- ✦ Aligned to satisfy every element's alignment requirement
- ✦ Pointer to first element
- ✦ No bounds checking

▣ **Structures**

- ✦ Allocate bytes in order declared
- ✦ Alignment

▣ **Unions**

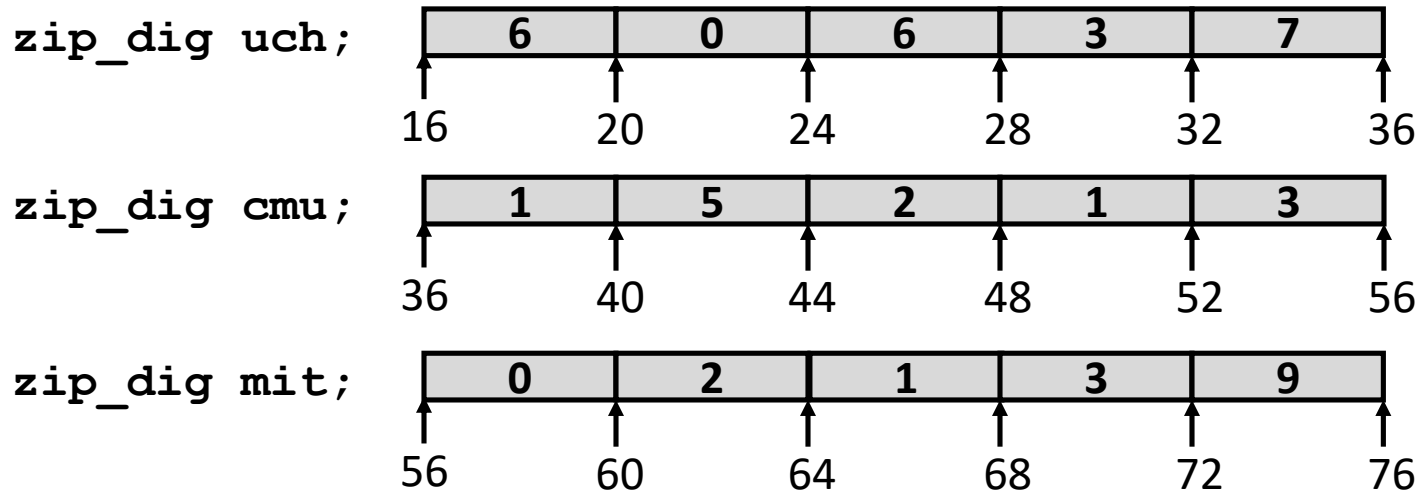
- ✦ Overlay declarations

Extra Materials

Array Example

```
#define ZLEN 5
typedef int zip_dig[ZLEN];

zip_dig uch = { 6, 0, 6, 3, 7 };
zip_dig cmu = { 1, 5, 2, 1, 3 };
zip_dig mit = { 0, 2, 1, 3, 9 };
```



- ❑ Declaration “`zip_dig uch`” equivalent to “`int uch[ZLEN]`”
- ❑ These arrays were allocated in successive 20 byte blocks
 - ☐ Not guaranteed to happen in general

Array Loop Example

```
void zincr(zip_dig z) {  
    size_t i;  
    for (i = 0; i < ZLEN; i++)  
        z[i]++;  
}
```

ZLEN = 5

```
# %rdi = z  
movq    $0, %rax          # i = 0 (init)  
jmp     .L3               # goto .L3  
.L4:                    # loop:  
    addl    $1, (%rdi,%rax,4) # z[i]++ (loop body)  
    addq    $1, %rax        # i++ (update)  
.L3:                    #  
    cmpq    $4, %rax       # i:4  
    jle     .L4            # if <=, goto loop (condition)  
ret
```

Array Loop Example

```
void zincr(zip_dig z) {  
    size_t i;  
    for (i = 0; i < ZLEN; i++)  
        z[i]++;  
}
```

ZLEN = 5

Array starting
address

Index (i)

```
# %rdi = z  
movq    $0, %rax          # i = 0 (init)  
jmp     .L3               # goto .L3  
.L4:                    # loop:  
addl    $1, (%rdi,%rax,4) # z[i]++ (loop body)  
addq    $1, %rax          # i++ (update)  
.L3:                    #  
cmpq    $4, %rax          # i:4  
jle     .L4               # if <=, goto loop (condition)  
ret
```